Final Report

Amna Malik

Caden Craddock

Matthew Reyna

Texas Tech University

CS2365: Object-Oriented Programming

4/26/2026

Table of Contents

Introduction 5

System Design Overview 6

UML / Class Diagram 7

OOP Design Concepts 8

- Abstraction / Encapsulation 8
- Interface 8
- Abstract Class 9
- Inheritance 9
- Exception Handling 9

GUI Design and User Interaction 11

Test Runs and Output 12

Limitations and Future Improvements 15

Conclusion 16

Appendices 17

- Account.java 17
- SavingsAccount.java 18
- ExpenseCategory.java 19
- Transaction.java 20
- BudgetManager.java 22

BudgetSummary.java 24

Trackable.java 25

InvalidBudgetException.java 25

BudgetTrackerGUI.java 26

Table of Figures

Figure 1: UML Class Diagram for Budget Tracker System 7

Figure 2: Budget Tracker GUI - Initial Transaction State 12

Figure 3: Budget Tracker GUI - After Transaction Deletion..... 13

Figure 4: Budget Tracker GUI - Updated Savings Goal and Budget..... 13

Figure 5: Monthly Summary Output Popup 14

Introduction

This project is a Personal Finance / Budget Tracker application developed using Java and JavaFX. The goal of the program is to help users manage their financial activity by tracking income and expenses, setting budget limits, and viewing a summary of their financial performance.

The application provides a graphical user interface (GUI) where users can interact with the system in real time. Users can add transactions, categorize spending, update budget limits, and monitor their financial status dynamically. This project demonstrates key object-oriented programming (OOP) principles, including abstraction, encapsulation, inheritance, interfaces, and exception handling.

This project focuses not only on functionality, but also on demonstrating clean software design using object-oriented principles.

System Design Overview

The program is structured into multiple classes, each responsible for a specific function:

- **Account** – base abstract class for financial accounts
- **SavingsAccount** – handles balance and savings goals
- **ExpenseCategory** – tracks spending and budget limits
- **Transaction** – represents individual income/expense entries
- **BudgetManager** – manages accounts and transactions
- **BudgetSummary** – calculates and formats financial summaries
- **Trackable** – interface for transaction tracking behavior
- **InvalidBudgetException** – handles invalid input cases
- **BudgetTrackerGUI** – provides the user interface

This modular design ensures separation of concerns and improves maintainability. Together, these components work to separate logic, data management, and user interaction, creating a well-organized and maintainable system.

UML / Class Diagram

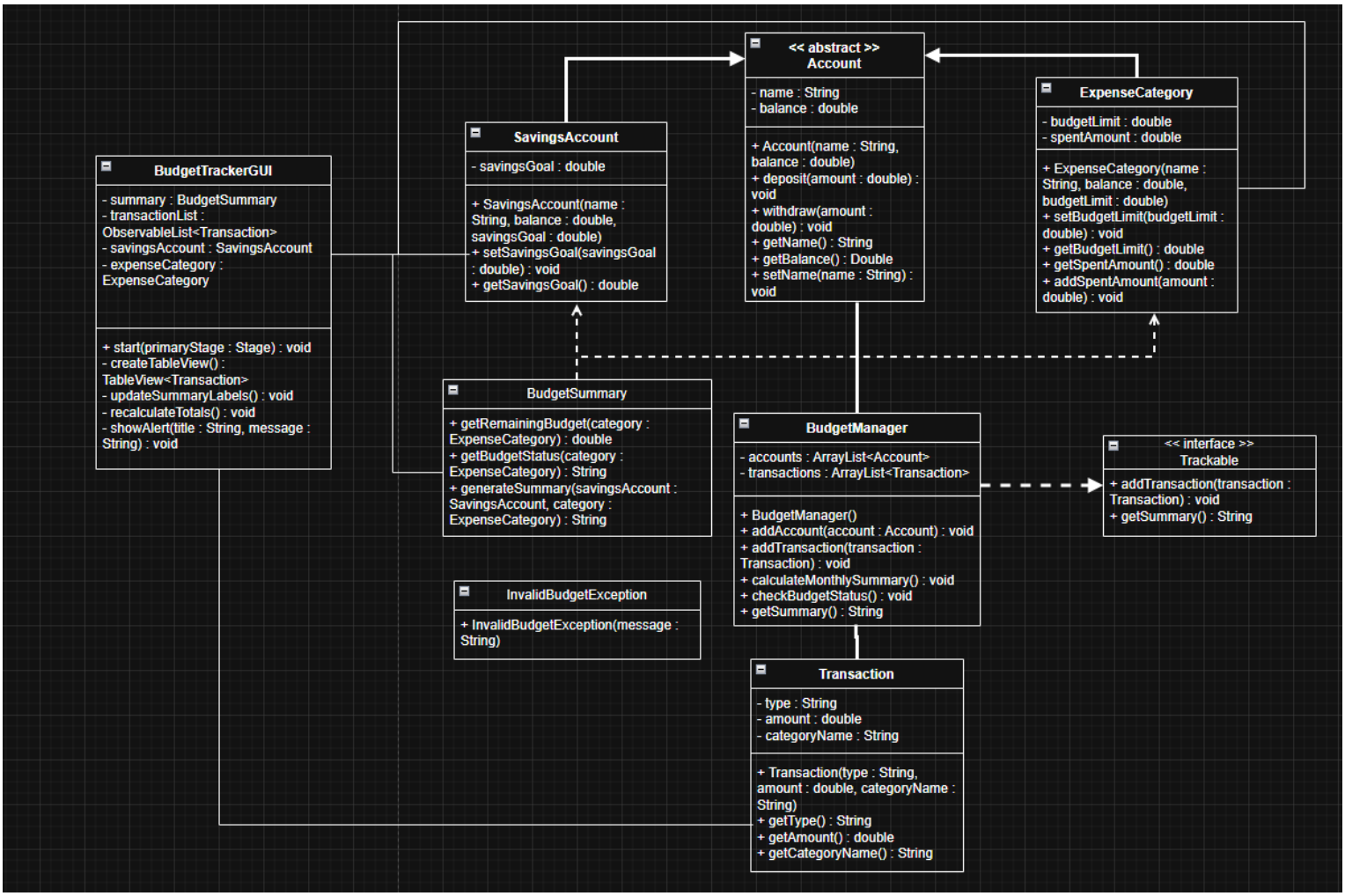


Figure 1: UML Class Diagram for Budget Tracker System

OOP Design Concepts

Abstraction / Encapsulation

Abstraction is implemented by modeling real-world financial components into separate classes such as **Account**, **Transaction**, **ExpenseCategory**, and **BudgetSummary**. Each class is responsible for a specific task. For example, the Transaction class represents a single financial entry, while **BudgetSummary** is responsible for computing and displaying financial results.

Encapsulation is enforced by declaring all class variables as private and providing public methods to access or modify them. For instance, the **Account** class stores balance as a private variable and updates it only through controlled methods like **deposit()** and **withdraw()**. This prevents direct modification of sensitive data and ensures consistency across the program.

Additionally, validation logic inside constructors (such as in the **Transaction** class) ensures that invalid data cannot be stored in objects, further strengthening encapsulation.

Interface

The Trackable interface defines a contract for tracking functionality by requiring implementing classes to provide methods such as **addTransaction()** and **getSummary()**.

This allows the **BudgetManager** class to follow a consistent structure while implementing its own logic for handling transactions. By using an interface, the design remains flexible and can be extended in the future to support additional tracking classes without modifying existing code.

This demonstrates abstraction by separating what the program should do from how it is actually implemented. It allows different classes to follow the same structure while handling their own logic. This makes the design more flexible and easier to expand in the future.

Abstract Class

The **Account** class is designed as an abstract class to represent a general financial account. It contains shared attributes such as name and balance, along with common methods like **deposit()** and **withdraw()**.

By making **Account** abstract, the program ensures that it is not instantiated directly, but instead serves as a base for more specific account types. This allows subclasses to inherit core functionality while specializing their behavior.

Inheritance

Inheritance is demonstrated through the **SavingsAccount** and **ExpenseCategory** classes, both of which extend the **Account** class.

- **SavingsAccount** adds functionality related to savings goals
- **ExpenseCategory** introduces budget tracking and spending limits

This design allows both classes to reuse the balance management logic from the parent class while adding their own specialized features. It reduces code duplication and creates a clear and logical hierarchy within the system.

Exception Handling

The project uses a custom exception class, **InvalidBudgetException**, to handle invalid user inputs in a structured way. This exception is thrown in the **Transaction** constructor when:

- the transaction type is empty
- the type is not “income” or “expense”
- the amount is zero or negative
- the category name is empty

By validating inputs at the object creation level, the program ensures that invalid data never enters the system.

Additionally, standard exceptions such as **NumberFormatException** are used in the GUI to handle cases where users input non-numeric values. These exceptions are caught and handled using alert messages, preventing crashes and improving the overall user experience.

GUI Design and User Interaction

The JavaFX GUI provides a user-friendly interface for interacting with the application. The layout is divided into two main sections:

- **Left Panel:** Input fields for transactions and action buttons
- **Right Panel:** Transaction table and financial summary

Key features include:

- Dropdown selection for transaction type
- TableView to display transactions
- Editable budget limit using a toggle (edit/save) button
- Real-time updates to balance and spending
- Popup summary display

The GUI improves usability and makes the program easier to understand compared to a console-based application. The GUI also allows users to edit the budget dynamically using an interactive toggle button, improving usability and user control.

JavaFX significantly improves understanding of the project by providing a visual and interactive representation of financial data. Instead of relying on console output, users can clearly see transactions, balances, and budget changes in real time, making the system easier to interpret. This makes the program more intuitive and user-friendly, especially for those unfamiliar with command-line interfaces.

Test Runs and Output

The following figures demonstrate the functionality of the Budget Tracker GUI under different test scenarios. These include adding transactions, deleting entries, updating budget and savings goals, and displaying a monthly summary. Each figure highlights how the system updates dynamically in response to user interaction.

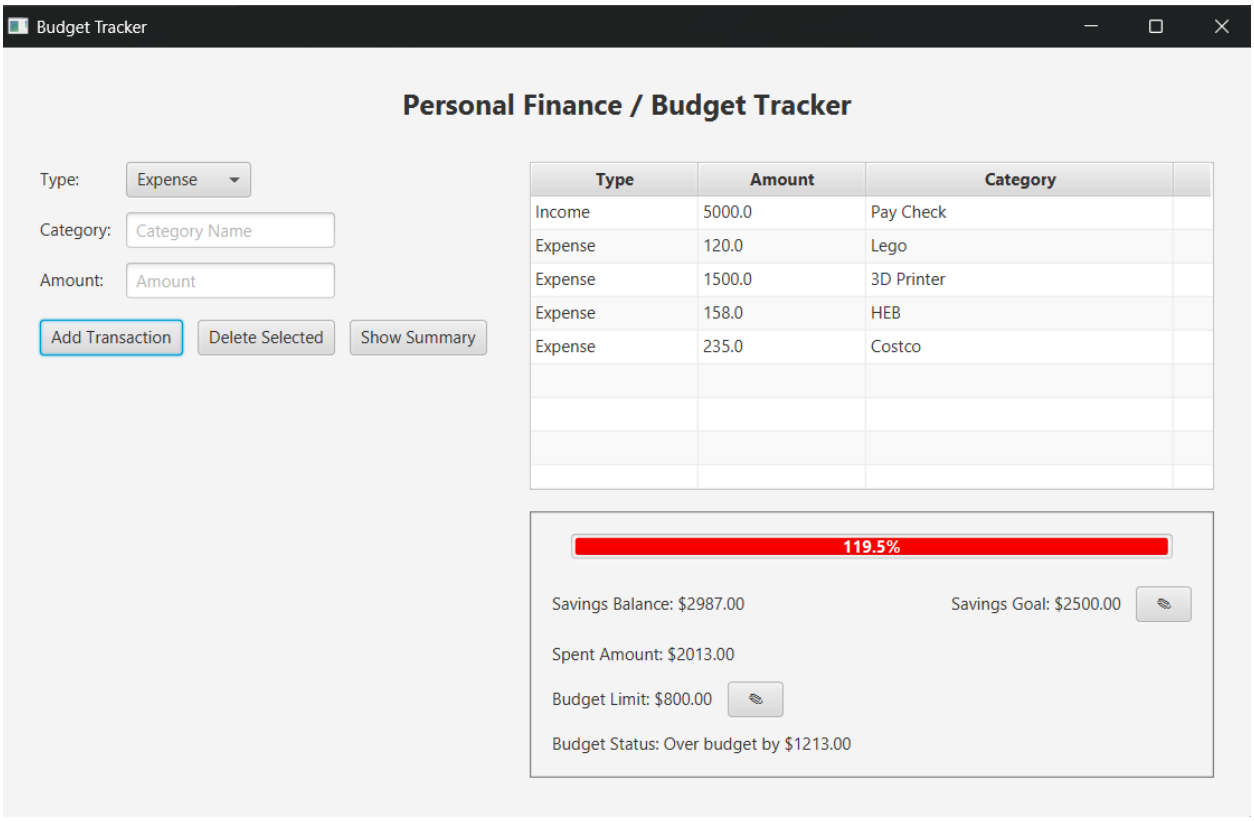


Figure 2: Budget Tracker GUI - Initial Transaction State

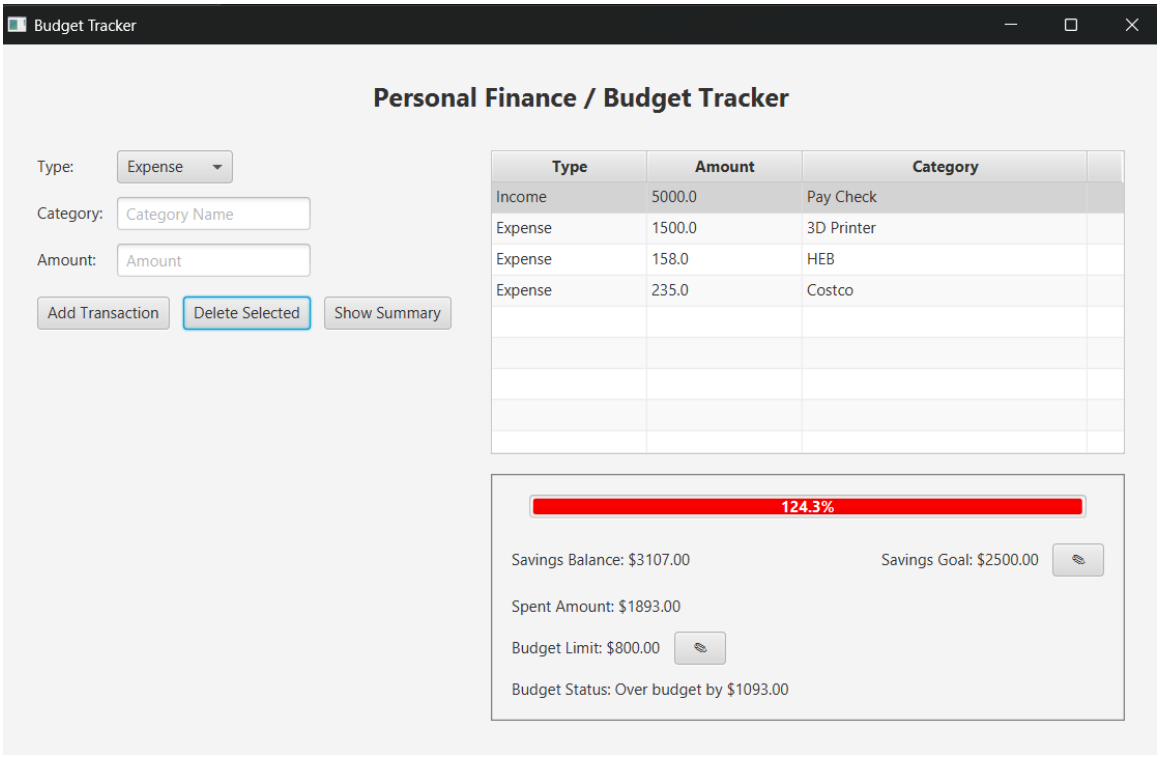


Figure 3: Budget Tracker GUI - After Transaction Deletion

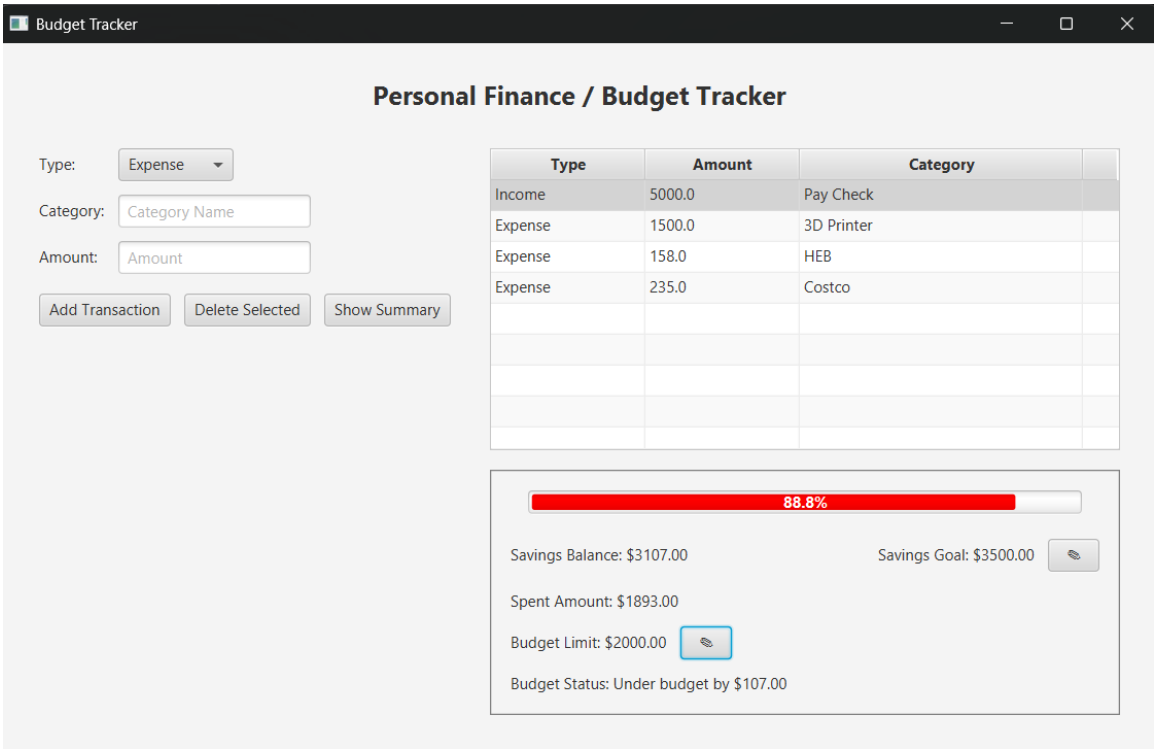


Figure 4: Budget Tracker GUI - Updated Savings Goal and Budget

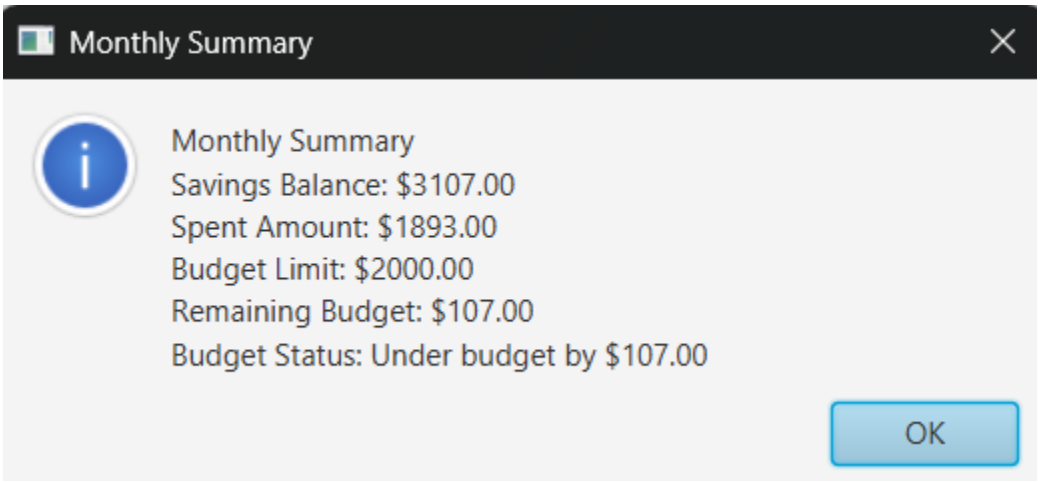


Figure 5: Monthly Summary Output Popup

The program was tested under multiple scenarios to ensure correctness:

- Valid income increases balance correctly
- Expenses decrease balance and increase spending
- Invalid inputs trigger error messages
- Deleting transactions updates totals accurately
- Budget editing updates budget status dynamically
- Summary output reflects correct financial data

The GUI successfully displays all changes in real time. For example, adding a $500 income followed by a $150 expense correctly updated the balance to $850 and correctly adjusted the budget status.

Limitations and Future Improvements

While the program meets all requirements, there are areas for improvement:

- Support for multiple accounts instead of a single account
- Ability to edit existing transactions
- Saving and loading data from files
- More advanced financial analytics (charts/graphs)
- Multiple budget categories instead of one general category

These enhancements would make the application more scalable and realistic.

Conclusion

This project successfully demonstrates the use of object-oriented programming concepts in a practical application. The system is modular, easy to understand, and interactive through the use of JavaFX.

By combining abstraction, encapsulation, inheritance, interfaces, and exception handling, the program achieves both functionality and maintainability. The GUI enhances the user experience and provides clear visualization of financial data.

Overall, this project meets all rubric requirements and provides a strong foundation for a real-world budgeting application. This project demonstrates how object-oriented design can be applied to solve real-world problems in a structured and scalable way.

Appendices

The following source files are included:

Account.java

```java
/**
* Represents a general financial account.
* This abstract class provides basic account functionality
* such as storing a name and balance, and performing deposits and
withdrawals.
*/
public abstract class Account {

   /** Stores the account name */
   private String name;

   /** Stores the account balance */
   private double balance;

   /**
    * Constructs an Account with a name and initial balance.
    *
    * @param name the name of the account
    * @param balance the starting balance of the account
    */
   public Account(String name, double balance) {
       this.name = name;
       this.balance = balance;
   }

   /**
    * Adds money to the account balance.
    *
    * @param amount the amount to deposit
    */
   public void deposit(double amount) {
       balance += amount;
   }

   /**
    * Subtracts money from the account balance.
    *
    * @param amount the amount to withdraw
    */
   public void withdraw(double amount) {
       balance -= amount;
   }

   /**
    * Returns the account name.
    *
    * @return the account name
```

```
     */
    public String getName() {
        return name;
    }

    /**
     * Returns the current account balance.
     *
     * @return the account balance
     */
    public double getBalance() {
        return balance;
    }

    /**
     * Updates the account name.
     *
     * @param name the new account name
     */
    public void setName(String name) {
        this.name = name;
    }
}
```

SavingsAccount.java

```
/**
* Represents a savings account.
* This class extends Account and adds a savings goal.
*/
public class SavingsAccount extends Account {

    /** Stores the savings goal for this account */
    private double savingsGoal;

    /**
     * Constructs a SavingsAccount with a name, balance, and savings goal.
     *
     * @param name the name of the savings account
     * @param balance the starting balance
     * @param savingsGoal the savings goal amount
     */
    public SavingsAccount(String name, double balance, double savingsGoal) {
        super(name, balance); // Calls parent Account constructor
        this.savingsGoal = savingsGoal;
    }

    /**
     * Updates the savings goal.
     *
     * @param savingsGoal the new savings goal amount
     */
    public void setSavingsGoal(double savingsGoal) {
        this.savingsGoal = savingsGoal;
    }
```

```
    /**
     * Returns the current savings goal.
     *
     * @return the savings goal
     */
    public double getSavingsGoal() {
        return savingsGoal;
    }
}
```

ExpenseCategory.java

```
/**
* Represents an expense category used for budget tracking.
* This class extends Account and adds budget limit and spending tracking.
*/
public class ExpenseCategory extends Account {

    /** Stores the budget limit for this category */
    private double budgetLimit;

    /** Stores how much has been spent so far */
    private double spentAmount;

    /**
     * Constructs an ExpenseCategory with a name, balance, and budget limit.
     *
     * @param name the category name
     * @param balance the starting balance
     * @param budgetLimit the maximum allowed spending
     */
    public ExpenseCategory(String name, double balance, double budgetLimit) {
        super(name, balance);    // Calls parent Account constructor
        this.budgetLimit = budgetLimit;
        this.spentAmount = 0.0; // Start with no spending
    }

    /**
     * Updates the budget limit.
     *
     * @param budgetLimit the new budget limit
     */
    public void setBudgetLimit(double budgetLimit) {
        this.budgetLimit = budgetLimit;
    }

    /**
     * Returns the current budget limit.
     *
     * @return the budget limit
     */
    public double getBudgetLimit() {
        return budgetLimit;
```

```
    }

    /**
     * Returns how much has been spent.
     *
     * @return the total spent amount
     */
    public double getSpentAmount() {
        return spentAmount;
    }

    /**
     * Adds to the total spent amount.
     *
     * @param amount the amount to add to spending
     */
    public void addSpentAmount(double amount) {
        spentAmount += amount;
    }
}
```

Transaction.java

```
/**
* Represents a single income or expense transaction.
* Includes validation to ensure all transaction data is valid.
*/
public class Transaction {

    /** Stores the type of transaction (Income or Expense) */
    private String type;

    /** Stores the transaction amount */
    private double amount;

    /** Stores the category name (e.g., Food, Rent) */
    private String categoryName;

    /**
     * Constructs a Transaction with validation.
     *
     * @param type the type of transaction (income or expense)
     * @param amount the transaction amount
     * @param categoryName the category of the transaction
     * @throws InvalidBudgetException if any input is invalid
     */
    public Transaction(String type, double amount, String categoryName)
            throws InvalidBudgetException {

        // Check if transaction type is empty
        if (type == null || type.trim().isEmpty()) {
            throw new InvalidBudgetException("Transaction type cannot be
empty.");
        }
```

```
        // Check if type is valid (must be income or expense)
        if (!type.equalsIgnoreCase("income") &&
!type.equalsIgnoreCase("expense")) {
            throw new InvalidBudgetException("Transaction type must be income
or expense.");
        }

        // Check if amount is valid (must be greater than 0)
        if (amount <= 0) {
            throw new InvalidBudgetException("Transaction amount must be
greater than zero.");
        }

        // Check if category name is empty
        if (categoryName == null || categoryName.trim().isEmpty()) {
            throw new InvalidBudgetException("Category name cannot be
empty.");
        }

        // Assign values after validation passes
        this.type = type;
        this.amount = amount;
        this.categoryName = categoryName;
    }

    /**
     * Returns the transaction type.
     *
     * @return the transaction type
     */
    public String getType() {
        return type;
    }

    /**
     * Returns the transaction amount.
     *
     * @return the transaction amount
     */
    public double getAmount() {
        return amount;
    }

    /**
     * Returns the category name.
     *
     * @return the category name
     */
    public String getCategoryName() {
        return categoryName;
    }
}
```

BudgetManager.java

```
import java.util.ArrayList;

/**
* Manages accounts and transactions.
* Implements the Trackable interface to handle transaction tracking
* and provide summary information.
*/
public class BudgetManager implements Trackable {

   /** List to store all accounts */
   private ArrayList<Account> accounts;

   /** List to store all transactions */
   private ArrayList<Transaction> transactions;

   /**
    * Constructs a BudgetManager with empty account and transaction lists.
    */
   public BudgetManager() {
       accounts = new ArrayList<>();
       transactions = new ArrayList<>();
   }

   /**
    * Adds an account to the list.
    *
    * @param account the account to add
    * @throws InvalidBudgetException if the account is null
    */
   public void addAccount(Account account) throws InvalidBudgetException {

       if (account == null) {
           throw new InvalidBudgetException("Account cannot be null.");
       }

       accounts.add(account);
   }

   /**
    * Adds a transaction and updates the account balance.
    *
    * @param transaction the transaction to add
    * @throws InvalidBudgetException if the transaction is null
    */
   @Override
   public void addTransaction(Transaction transaction) throws
InvalidBudgetException {

       if (transaction == null) {
           throw new InvalidBudgetException("Transaction cannot be null.");
       }
```

```
        transactions.add(transaction);

        // Update balance based on transaction type
        if (transaction.getType().equalsIgnoreCase("income")) {
            accounts.get(0).deposit(transaction.getAmount());
        } else if (transaction.getType().equalsIgnoreCase("expense")) {
            accounts.get(0).withdraw(transaction.getAmount());
        }
    }

    /**
     * Calculates and prints the monthly income, expenses, and net total.
     */
    public void calculateMonthlySummary() {

        double totalIncome = 0;
        double totalExpenses = 0;

        for (Transaction transaction : transactions) {
            if (transaction.getType().equalsIgnoreCase("income")) {
                totalIncome += transaction.getAmount();
            } else if (transaction.getType().equalsIgnoreCase("expense")) {
                totalExpenses += transaction.getAmount();
            }
        }

        System.out.println("Total Income: $" + totalIncome);
        System.out.println("Total Expenses: $" + totalExpenses);
        System.out.println("Net Total: $" + (totalIncome - totalExpenses));
    }

    /**
     * Checks the budget status.
     * (Currently a placeholder for future implementation.)
     */
    public void checkBudgetStatus() {
        System.out.println("Budget status checked.");
    }

    /**
     * Returns a summary of the number of accounts and transactions.
     *
     * @return summary string
     */
    @Override
    public String getSummary() {
        return "Accounts: " + accounts.size() +
                "\nTransactions: " + transactions.size();
    }
}
```

BudgetSummary.java

```
/**
* Handles budget calculations and generates summary output.
* Provides methods to calculate remaining budget, determine budget status,
* and generate a formatted monthly summary.
*/
public class BudgetSummary {

   /**
    * Calculates how much budget is remaining.
    *
    * @param category the expense category
    * @return remaining budget amount
    */
   public double getRemainingBudget(ExpenseCategory category) {
       return category.getBudgetLimit() - category.getSpentAmount();
   }

   /**
    * Determines whether the user is over or under budget.
    *
    * @param category the expense category
    * @return a string indicating budget status
    */
   public String getBudgetStatus(ExpenseCategory category) {

       // Get remaining budget
       double remaining = getRemainingBudget(category);

       // If negative, user is over budget
       if (remaining < 0) {
           return "Over budget by $" + String.format("%.2f",
Math.abs(remaining));
       }
       // Otherwise, user is under budget
       else {
           return "Under budget by $" + String.format("%.2f", remaining);
       }
   }

   /**
    * Generates a full monthly summary as a formatted string.
    *
    * @param savingsAccount the savings account
    * @param category the expense category
    * @return formatted summary string
    */
   public String generateSummary(SavingsAccount savingsAccount,
ExpenseCategory category) {

       return "Monthly Summary\n"
               // Shows current savings balance
               + "Savings Balance: $" + String.format("%.2f",
savingsAccount.getBalance()) + "\n"
```

```
                // Shows total amount spent
                + "Spent Amount: $" + String.format("%.2f",
category.getSpentAmount()) + "\n"

                // Shows budget limit
                + "Budget Limit: $" + String.format("%.2f",
category.getBudgetLimit()) + "\n"

                // Shows remaining budget
                + "Remaining Budget: $" + String.format("%.2f",
getRemainingBudget(category)) + "\n"

                // Shows budget status (over/under)
                + "Budget Status: " + getBudgetStatus(category);
    }
}
```

Trackable.java

```
/**
* Defines tracking behavior for classes that manage transactions.
* Any class that implements this interface must provide methods
* to add transactions and generate a summary.
*/
public interface Trackable {

    /**
     * Adds a transaction to the system.
     *
     * @param transaction the transaction to add
     * @throws InvalidBudgetException if the transaction is invalid
     */
    void addTransaction(Transaction transaction) throws
InvalidBudgetException;

    /**
     * Returns a summary of the tracked data.
     *
     * @return a summary string
     */
    String getSummary();
}
```

InvalidBudgetException.java

```
/**
* Custom exception used for invalid budget-related inputs.
* This exception is thrown when user input violates validation rules.
*/
public class InvalidBudgetException extends Exception {

    /**
```

```
    * Constructs an InvalidBudgetException with a specific error message.
    *
    * @param message the error message explaining what went wrong
    */
   public InvalidBudgetException(String message) {

       // Calls the parent Exception class constructor
       super(message);
   }
}
```

BudgetTrackerGUI.java

```
import javafx.application.Application;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import javafx.stage.Stage;

/**
 * Main JavaFX class that creates the Budget Tracker window.
 * This class handles:
 * - Adding income and expense transactions
 * - Deleting selected transactions
 * - Editing the savings goal
 * - Editing the budget limit
 * - Showing savings progress with a progress bar
 * - Displaying budget summary information
 */
public class BudgetTrackerGUI extends Application {

    /** Object used to calculate budget status and monthly summaries */
    private BudgetSummary summary = new BudgetSummary();

    /** List that stores all transactions shown in the table */
    private final ObservableList<Transaction> transactionList =
FXCollections.observableArrayList();

    /** Main savings account object */
    private SavingsAccount savingsAccount;

    /** Expense category object used for budget tracking */
    private ExpenseCategory expenseCategory;

    /** Labels used to display summary information */
    private final Label balanceLabel = new Label();
    private final Label goalLabel = new Label();
    private final Label spentLabel = new Label();
    private final Label budgetLabel = new Label();
    private final Label statusLabel = new Label();
```

```
    /** Progress bar used to show savings progress */
    private final ProgressBar savingsProgressBar = new ProgressBar();

    /** Label displayed on top of the progress bar */
    private final Label progressPercentLabel = new Label();

    /**
     * Starts the JavaFX application and builds the main GUI window.
     *
     * @param primaryStage the main application window
     */
    @Override
    public void start(Stage primaryStage) {

        // Create starting account and budget data
        savingsAccount = new SavingsAccount("Main Savings", 0.0, 2000.0);
        expenseCategory = new ExpenseCategory("General Expenses", 0.0,
500.0);

        // Create title label
        Label titleLabel = new Label("Personal Finance / Budget Tracker");
        titleLabel.setStyle("-fx-font-size: 20px; -fx-font-weight: bold;");

        // Dropdown box for choosing income or expense
        ComboBox<String> typeBox = new ComboBox<>();
        typeBox.getItems().addAll("Income", "Expense");
        typeBox.setValue("Expense");

        // Text field for category name
        TextField categoryField = new TextField();
        categoryField.setPromptText("Category Name");

        // Text field for transaction amount
        TextField amountField = new TextField();
        amountField.setPromptText("Amount");

        // Buttons for adding, deleting, and showing summary
        Button addButton = new Button("Add Transaction");
        Button deleteButton = new Button("Delete Selected");
        Button summaryButton = new Button("Show Summary");

        // Hidden text field used when editing the savings goal
        TextField goalField = new TextField();
        goalField.setPromptText("New Savings Goal");
        goalField.setVisible(false);
        goalField.setManaged(false);

        // Pencil button used to edit the savings goal
        Button updateGoalButton = new Button("✎");
        updateGoalButton.setPrefWidth(40);
        updateGoalButton.setTooltip(new Tooltip("Edit Savings Goal"));

        // Hidden text field used when editing the budget limit
        TextField budgetField = new TextField();
        budgetField.setPromptText("New Budget Limit");
```

```
        budgetField.setVisible(false);
        budgetField.setManaged(false);

        // Pencil button used to edit the budget limit
        Button updateBudgetButton = new Button("✎");
        updateBudgetButton.setPrefWidth(40);
        updateBudgetButton.setTooltip(new Tooltip("Edit Budget"));

        // Create table and connect it to the transaction list
        TableView<Transaction> tableView = createTableView();
        tableView.setItems(transactionList);

        // Runs when the Add Transaction button is clicked
        addButton.setOnAction(e -> {
            try {
                String type = typeBox.getValue();
                String category = categoryField.getText().trim();
                String amountText = amountField.getText().trim();

                if (category.isEmpty()) {
                    showAlert("Input Error", "Please enter a category
name.");
                    return;
                }

                double amount = Double.parseDouble(amountText);

                if (amount <= 0) {
                    showAlert("Input Error", "Amount must be greater than
0.");
                    return;
                }

                // Create and add transaction to the table
                Transaction transaction = new Transaction(type, amount,
category);
                transactionList.add(transaction);

                // Income adds to savings, expenses subtract from savings
                if (type.equalsIgnoreCase("Income")) {
                    savingsAccount.deposit(amount);
                } else {
                    savingsAccount.withdraw(amount);
                    expenseCategory.addSpentAmount(amount);
                }

                updateSummaryLabels();

                categoryField.clear();
                amountField.clear();

            } catch (InvalidBudgetException ex) {
                showAlert("Input Error", ex.getMessage());
            } catch (NumberFormatException ex) {
                showAlert("Input Error", "Please enter a valid number.");
            }
```

```
        });

        // Runs when the edit savings goal button is clicked
        updateGoalButton.setOnAction(e -> {
            if (!goalField.isVisible()) {

                // Show text field so user can enter a new goal
                goalField.setVisible(true);
                goalField.setManaged(true);

goalField.setText(String.valueOf(savingsAccount.getSavingsGoal()));
                updateGoalButton.setText("✓");

            } else {
                try {
                    double newGoal =
Double.parseDouble(goalField.getText().trim());

                    if (newGoal <= 0) {
                        showAlert("Input Error", "Savings goal must be
greater than 0.");
                        return;
                    }

                    // Update savings goal
                    savingsAccount.setSavingsGoal(newGoal);
                    updateSummaryLabels();

                    // Hide field again after updating
                    goalField.setVisible(false);
                    goalField.setManaged(false);
                    updateGoalButton.setText("✎");

                } catch (NumberFormatException ex) {
                    showAlert("Input Error", "Please enter a valid savings
goal.");
                }
            }
        });

        // Runs when the edit budget button is clicked
        updateBudgetButton.setOnAction(e -> {
            if (!budgetField.isVisible()) {

                // Show text field so user can enter a new budget limit
                budgetField.setVisible(true);
                budgetField.setManaged(true);

budgetField.setText(String.valueOf(expenseCategory.getBudgetLimit()));
                updateBudgetButton.setText("✓");

            } else {
                try {
                    double newLimit =
Double.parseDouble(budgetField.getText().trim());
```

```
                    if (newLimit <= 0) {
                        showAlert("Input Error", "Budget limit must be
greater than 0.");
                        return;
                    }

                    // Update budget limit
                    expenseCategory.setBudgetLimit(newLimit);
                    updateSummaryLabels();

                    // Hide field again after updating
                    budgetField.setVisible(false);
                    budgetField.setManaged(false);
                    updateBudgetButton.setText("✎");

                } catch (NumberFormatException ex) {
                    showAlert("Input Error", "Please enter a valid budget
amount.");
                }
            }
        });

        // Runs when the Delete Selected button is clicked
        deleteButton.setOnAction(e -> {
            Transaction selected =
tableView.getSelectionModel().getSelectedItem();

            if (selected == null) {
                showAlert("Selection Error", "Please select a transaction to
delete.");
                return;
            }

            // Remove selected transaction and recalculate totals
            transactionList.remove(selected);
            recalculateTotals();
            updateSummaryLabels();
        });

        // Shows full monthly summary in a popup
        summaryButton.setOnAction(e -> {
            showAlert("Monthly Summary",
                    summary.generateSummary(savingsAccount,
expenseCategory));
        });

        // Create grid layout for input fields
        GridPane inputPane = new GridPane();
        inputPane.setHgap(10);
        inputPane.setVgap(10);
        inputPane.add(new Label("Type:"), 0, 0);
        inputPane.add(typeBox, 1, 0);
        inputPane.add(new Label("Category:"), 0, 1);
        inputPane.add(categoryField, 1, 1);
        inputPane.add(new Label("Amount:"), 0, 2);
        inputPane.add(amountField, 1, 2);
```

```
        // Put main action buttons in one row
        HBox buttonBox = new HBox(10, addButton, deleteButton,
summaryButton);
        buttonBox.setAlignment(Pos.CENTER_LEFT);

        // Left side contains input fields and buttons
        VBox leftPanel = new VBox(15, inputPane, buttonBox);
        leftPanel.setPadding(new Insets(15));

        // Put savings goal label and edit button together
        HBox goalEditRow = new HBox(10, goalLabel, updateGoalButton);
        goalEditRow.setAlignment(Pos.CENTER_RIGHT);

        // Put budget label and edit button together
        HBox budgetRow = new HBox(10, budgetLabel, updateBudgetButton);
        budgetRow.setAlignment(Pos.CENTER_LEFT);

        // Style the savings progress bar
        savingsProgressBar.setPrefWidth(430);
        savingsProgressBar.setPrefHeight(25);
        savingsProgressBar.setStyle("-fx-accent: red;");

        // Style progress text
        progressPercentLabel.setStyle("-fx-text-fill: white; -fx-font-weight:
bold;");

        // StackPane places the percentage label on top of the progress bar
        StackPane progressStack = new StackPane();
        progressStack.getChildren().addAll(savingsProgressBar,
progressPercentLabel);

        // This row keeps savings balance on the left and savings goal on the
right
        HBox balanceGoalRow = new HBox();
        balanceGoalRow.setAlignment(Pos.CENTER);
        balanceGoalRow.setPadding(new Insets(8, 0, 5, 0));

        Region spacer = new Region();
        HBox.setHgrow(spacer, Priority.ALWAYS);

        balanceGoalRow.getChildren().addAll(balanceLabel, spacer,
goalEditRow);

        // Progress section goes at the top of the summary box
        VBox progressBox = new VBox(10, progressStack, balanceGoalRow);
        progressBox.setAlignment(Pos.CENTER);

        // Summary panel displays progress, spending, budget, and status
        VBox summaryBox = new VBox(10,
                progressBox,
                goalField,
                spentLabel,
                budgetRow,
                budgetField,
                statusLabel
        );
```

```
        summaryBox.setPadding(new Insets(15));
        summaryBox.setStyle("-fx-border-color: gray; -fx-border-width: 1;");

        // Right side contains table and summary panel
        VBox rightPanel = new VBox(15, tableView, summaryBox);
        rightPanel.setPadding(new Insets(15));
        VBox.setVgrow(tableView, Priority.ALWAYS);

        // Display starting summary values after GUI components are created
        updateSummaryLabels();

        // Main layout for the full window
        BorderPane root = new BorderPane();
        root.setTop(wrapTop(titleLabel));
        root.setLeft(leftPanel);
        root.setCenter(rightPanel);
        root.setPadding(new Insets(15));

        // Create and show the window
        Scene scene = new Scene(root, 900, 550);
        primaryStage.setTitle("Budget Tracker");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    /**
     * Creates the table used to display income and expense transactions.
     *
     * @return the completed transaction table
     */
    private TableView<Transaction> createTableView() {
        TableView<Transaction> table = new TableView<>();

        // Type column shows Income or Expense
        TableColumn<Transaction, String> typeCol = new TableColumn<>("Type");
        typeCol.setCellValueFactory(new PropertyValueFactory<>("type"));
        typeCol.setPrefWidth(120);

        // Amount column shows the transaction amount
        TableColumn<Transaction, Double> amountCol = new
TableColumn<>("Amount");
        amountCol.setCellValueFactory(new PropertyValueFactory<>("amount"));
        amountCol.setPrefWidth(120);

        // Category column shows the transaction category name
        TableColumn<Transaction, String> categoryCol = new
TableColumn<>("Category");
        categoryCol.setCellValueFactory(new
PropertyValueFactory<>("categoryName"));
        categoryCol.setPrefWidth(220);

        table.getColumns().addAll(typeCol, amountCol, categoryCol);

        return table;
    }
```

```
    /**
     * Updates the savings progress bar and all summary labels.
     * This method is called after adding, deleting, or editing values.
     */
    private void updateSummaryLabels() {
        double balance = savingsAccount.getBalance();
        double goal = savingsAccount.getSavingsGoal();

        // Calculate savings progress as a decimal value
        double progress = 0.0;

        if (goal > 0) {
            progress = balance / goal;
        }

        // ProgressBar only accepts values from 0.0 to 1.0
        savingsProgressBar.setProgress(Math.min(progress, 1.0));

        // Update progress bar text
        progressPercentLabel.setText(String.format("%.1f%%", progress *
100));

        // Update regular summary labels
        balanceLabel.setText(String.format("Savings Balance: $%.2f",
balance));
        goalLabel.setText(String.format("Savings Goal: $%.2f", goal));
        spentLabel.setText(String.format("Spent Amount: $%.2f",
expenseCategory.getSpentAmount()));
        budgetLabel.setText(String.format("Budget Limit: $%.2f",
expenseCategory.getBudgetLimit()));
        statusLabel.setText("Budget Status: " +
summary.getBudgetStatus(expenseCategory));
    }

    /**
     * Recalculates account balance and spending after a transaction is
deleted.
     * This keeps totals correct because removing a transaction should also
undo
     * its effect on the savings balance and spent amount.
     */
    private void recalculateTotals() {
        double currentBudgetLimit = expenseCategory.getBudgetLimit();
        double currentSavingsGoal = savingsAccount.getSavingsGoal();

        // Reset account and category while keeping edited goal and budget
values
        savingsAccount = new SavingsAccount("Main Savings", 0.0,
currentSavingsGoal);
        expenseCategory = new ExpenseCategory("General Expenses", 0.0,
currentBudgetLimit);

        // Add each remaining transaction back into the totals
        for (Transaction transaction : transactionList) {
            if (transaction.getType().equalsIgnoreCase("Income")) {
                savingsAccount.deposit(transaction.getAmount());
            } else {
```

```
                savingsAccount.withdraw(transaction.getAmount());
                expenseCategory.addSpentAmount(transaction.getAmount());
            }
        }
    }

    /**
     * Centers the title label at the top of the window.
     *
     * @param titleLabel the title label to display
     * @return an HBox containing the centered title
     */
    private HBox wrapTop(Label titleLabel) {
        HBox box = new HBox(titleLabel);
        box.setAlignment(Pos.CENTER);
        box.setPadding(new Insets(10));
        return box;
    }

    /**
     * Displays a popup message for errors or summaries.
     *
     * @param title the alert window title
     * @param message the alert message text
     */
    private void showAlert(String title, String message) {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle(title);
        alert.setHeaderText(null);
        alert.setContentText(message);
        alert.showAndWait();
    }

    /**
     * Launches the JavaFX application.
     *
     * @param args command-line arguments
     */
    public static void main(String[] args) {
        launch(args);
    }
}
```